REPORT · 2026年10月26日

H3D 中小型文物三维建模自动化系统 · v1.1.10（r14）发行说明

v1.1.10（r14）发行说明

H3D 项目组
2026年10月26日

目 录

一、发行概览 4

二、本版改动总表 6

三、缺陷①：窗口一跑就跳 7

3.1 现象：越跑越胖，底部被挤出屏幕 . . . 7

3.2 根因：布局的最小尺寸是「文字宽度」算出来的 . . . 7

3.3 修法：会变的文字一律不许参与宽度计算 . . . 7

3.4 屏幕自适应：先问屏幕，再定尺寸 . . . 7

3.5 验证：把文字换成 240 倍长串，窗口尺寸必须不变 . . . 8

四、缺陷②：约束没确认就开跑 9

4.1 三个问题合在一起 . . . 9

4.2 约束数据模型：八项，可序列化，可校验 . . . 9

4.3 闸门：不确认就不启动 . . . 9

4.4 运行期只读：只读卡里不许有输入控件 . . . 10

4.5 落盘：子进程只认 yaml . . . 10

五、缺陷③：阶段耗时只活在报表里 11

5.1 两条数据源，互补使用 . . . 11

5.2 显示：十枚 chip，5 列 × 2 行 . . . 11

5.3 口径：同一阶段多件求和，与墙钟分开标注 . . . 11

六、缺陷④：目录太散 13

6.1 旧布局的问题不是丑，是不知道该给谁 . . . 13

6.2 一行代码的代价：让 task.root 下沉一级 . . . 13

6.3 兼容：判据不能是「00_raw 在不在」 . . . 13

6.4 实测目录树与体积 . . . 14

七、两起事故复盘（本版顺手修掉） 15

7.1 事故 A：迁移逻辑吃掉了仓库的 output 目录 . . . 15

7.2 事故 B：PySide6 在无 windowHandle 时询问 screen 会堆损坏 15
7.3 事故 C：全跳过的件卡在 RUNNING，界面误报「运行未完成」 15
八、验证 **17**
8.1 单元测试：843 项全绿 17
8.2 端到端：CLI 跑通 + 目录树核对 + 报表口径 17
8.3 发行包 17
九、已知限制与下一步 **18**

一、发行概览

本次发行版本 **v1.1.10（修订 r14）**，打包日期 2026-10-26。与前几版不同，这一版**没有改一行算法**：SfM、配准、泊松、UV、纹理采样的代码全部未动。它改的是**人在用这个软件时所感知到的东西**——窗口会不会自己跳、开跑前知不知道自己选了什么、跑完能不能看出哪一步慢、算出来的东西被放在哪。

触发这一版的是一次真机使用。用户的反馈是四句话，对应四个真实缺陷：① 「运行的时候窗口时不时异常变动，进度看不全」；② 「选完照片就直接跑了，我根本不知道这次是不是稠密、精度几档」；③ 「总时间有，但每个阶段花了多久看不到，结束就没了」；④ 「生成的文件夹太多太散，不知道该拿哪个」。这四条有一个共同点：**管线本身是对的，但它没有把「正在做什么、按什么做、做完了放在哪」讲清楚。**

> **一句话结论**：本版把界面从**算法的显示层**提升为**一次生成的事前约定 + 事中观测 + 事后归置**——开跑前用一张约束确认框把「这次要什么」钉死（不确认不启动）；运行期把已确认的约束**只读展示**、把十个阶段的耗时**常驻**在总时长旁边；结束后所有产物收进**一个综合文件夹的两棵子树**（阶段成果 / 交付成果），交付时只需要打开其中一棵。



图 1 本版四项改动的因果链：全部由真机使用反馈驱动，未触及算法层

二、本版改动总表

#	问题（用户视角）	根因	改法	验证方式
1	运行中窗口自己变大变窄，底部状态被挤出屏幕	布局最小尺寸 = 运行时文字宽度；QProgressBar 的格式串同样计入 sizeHint；窗口初始尺寸写死 1280×800	ElidedLabel（最小宽 0、sizeHint 按 min_chars、全文进 tooltip）；进度条只显示百分比；左侧配置区进 QScrollArea；按屏幕可用区 94%/92% 定型并给最小尺寸封顶	把状态文字换成 240 倍长串，断言窗口宽高**一点不变**（15 项窗口用例）
2	选完照片直接跑，不知道这次的稠密/精度/纹理/格式	参数随时可改、随改随生效；默认值静默生效，跑完才发现几何是稀疏插值	ConstraintsDialog 闸门：不确认不启动；三键语义（开始生成 / 全部用默认值 / 取消）；确认结果落 config.yaml 的 runner.constraints + constraints_confirmed；运行期换成只读约束卡	取消分支断言「未进入自检/启动」；只读卡内 findChildren 四类输入控件 == []
3	总时间有，各阶段分别花了多久看不到；跑完就归零	阶段耗时只存在于报表，界面没有位置；运行期事件按阶段去重，多件批处理只记第一件	StageTimingStrip：十枚 chip 常驻（5×2）；运行中按「▲ 开始 / √ 完成」累计墙钟，结束后用 manifest 的 elapsed_s 覆盖（与报表同源）；同一阶段多件**求和**	timing 单测 + manifest 汇总口径用例；演示项目实测 S1 51.78 s / M4 11.06 s
4	文件夹太多太散，不知道该拿哪个	阶段中间态、交付模型、报表全平铺在项目根；80_export 与 report 是历史遗留的两处出口	core/paths.py 作为布局唯一出口：综合文件夹内分「阶段成果」（12 个阶段子目录）与「交付成果」（按文物编号 + 单一 report 目录）；GUI/CLI/runner/report 全部改走它	布局单测 24 项 + 端到端复跑；旧项目打开时自动迁移（带闸门）

四项改动共涉及 **18 个既有文件、659 行新增 / 140 行删除**，外加 4 个新模块与 3 个新测试文件（合计 1,869 行）：

新增文件	行数	职责
src/h3d/core/paths.py	388	输出布局唯一出口：两棵子树、路径回退链、旧布局迁移
src/h3d/gui/constraints.py	430	约束数据模型 + 确认对话框 + 只读摘要卡
src/h3d/gui/timing.py	222	阶段耗时条 + manifest 汇总
src/h3d/gui/widgets.py	98	ElidedLabel：不撑宽布局的省略标签
tests/test_paths_layout.py	277	布局 / 回退链 / 迁移闸门
tests/test_gui_constraints.py	234	约束模型 / 配置集成 / 启动闸门
tests/test_gui_window.py	220	窗口尺寸稳定性 + 阶段耗时展示

三、缺陷①：窗口一跑就跳

3.1 现象：越跑越胖，底部被挤出屏幕

用户描述是「窗口时不时异常变动」。复现后的具体表现是：不运行时窗口宽 1,360 px，一开始跑，状态行出现「● S1 运行中 · 深度图 12/30 · 最后输出 45 s 前」这类长句，窗口就变宽；等这句变短，窗口**不会缩回去**。笔记本 1366×768、150% 缩放下可用区只有 1280×656，被撑高之后底部的阶段灯、结果条与日志直接落到屏幕外——用户说的「进度看不全」，其实是**窗口放不下**。

第二个现象与文字无关：窗口初始尺寸是写死的 `resize(1280, 800)`。在可用高只有 656 px 的屏幕上，窗口**构造完成的那一刻就已经出屏**，用户看到的第一帧就是残缺的。而最小尺寸没有封顶，用户想手动拉小也拉不动。

3.2 根因：布局的最小尺寸是「文字宽度」算出来的

Qt 的布局系统用 `minimumSizeHint()` 决定一个控件至少要占多宽。对 `QLabel` 来说，它的最小值就是**整段文字的宽度**。于是运行时文字一变长，布局就要求更多空间，而顶层窗口没有固定宽度 → Qt 直接把窗口撑大；文字变短时窗口不会自动缩回（没有反向约束）。

还有一处更隐蔽：`QProgressBar` 的格式串（`setFormat("%p% · 第 k/N 件 · S1")`）**也参与 sizeHint 计算**。开发者通常以为那只是画在条上的字，实际上它同样会把整个窗口顶宽。本版之前的实现正好把件数与阶段编号塞进了格式串，属于同一个缺陷的第二发子弹。

3.3 修法：会变的文字一律不许参与宽度计算

新增 `ElidedLabel`（`src/h3d/gui/widgets.py`），三处改动：① `minimumSizeHint()` 的宽恒为 **0**——不管文字多长，布局都不会因此变宽；② `sizeHint()` 按 `min_chars` 估算一个固定宽度，所以长文本与短文本的自然宽度**完全相等**；③ 省略显示不等于丢信息——全文同步写进 `toolTip()`，自定义 `paintEvent` 用 `QFontMetrics.elidedText` 画中间省略号。

```
class ElidedLabel(QLabel):
    # 宽度不随文字变化的标签（防止运行时把窗口撑大）

    def minimumSizeHint(self):              # noqa: N802 - Qt 命名
        # 关键：宽为 0。文字再长也不向布局索取空间。
        return QSize(0, super().minimumSizeHint().height())

    def sizeHint(self):                     # noqa: N802 - Qt 命名
        fm = self.fontMetrics()
        w = fm.horizontalAdvance("0") * max(4, self._min_chars)
        return QSize(w, super().sizeHint().height())

    def setText(self, text):                # noqa: N802 - Qt 命名
        super().setText(text)
        self.setToolTip(text)               # 省略显示 != 丢信息
        self.update()
```

配套两处收口：进度条改回 `setFormat("%p%")`，件数与当前阶段挪到进度条右侧的 `ElidedLabel`（`lbl_count`）；实时状态行 `lbl_state` 与总时长 `lbl_elapsed` 同样换成 `ElidedLabel`。左侧配置区整体塞进 `QScrollArea`——这样展开「高级参数」也不会把窗口顶高，而是出现滚动条。

3.4 屏幕自适应：先问屏幕，再定尺寸

`_fit_to_screen()` 按 `availableGeometry()` 取**可用区**（已扣掉任务栏），窗口占 94% 宽 / 92% 高、居中显示，并且**把最小尺寸也封顶在可用区之内**。最后一条容易被忽略：最小尺寸若不封顶，布局会把窗口反过来撑到最小值以上，用户就再也拉不动了。

首次显示时再算一次（`showEvent` 里的 `_fitted` 标志）——多显示器与 DPI 缩放在构造期未必就绪。另外，阶段状态灯从十个横排改成 **5 列 × 2 行**：1366 px 宽的屏上，横排时每个灯只剩 40 px，代号看得见、状态看不清。

3.5 验证：把文字换成 240 倍长串，窗口尺寸必须不变

这类回归只测「能创建窗口」是抓不到的——旧实现在构造时完全正常，只有跑起来才跳。所以用例直接比对尺寸：

```
w = MainWindow(cfg); w.show()
before = (w.width(), w.height())
for n in (1, 60, 240):
    w.lbl_state.setText("● S1 运行中 · " + "深度图 12/30 " * n)
    w.lbl_elapsed.setText("总 01:23:45 · S1 45:00 · " + "x" * n)
    w.lbl_count.setText("第 12/34 件 · S1")
    w._on_progress(68)
    app.processEvents()
    assert (w.width(), w.height()) == before   # 一点都不能变
```

另有两条守门用例：`ElidedLabel.minimumSizeHint().width() == 0`、`long.sizeHint().width() == short.sizeHint().width()`；以及「进度条格式串必须短」——防止后来者又把长文本塞回 `setFormat`。

四、缺陷②：约束没确认就开跑

4.1 三个问题合在一起

旧界面的参数是**随时可改、改了立即生效**的。这带来三个问题：① 用户点了「一键生成交付物」，管线按当时控件的值跑，跑到一半想改「其实我想要 4096 图集」只能取消重来；② 更常见的是**根本没意识到自己选了什么**——默认关着稠密重建的纯照片项目跑完 30 分钟，才发现几何是稀疏点云插值出来的；③ 界面上的某些控件**改了不生效**（没有回写通道的假控件），用户以为改了，实际跑的是旧值，而且没有任何告警。

4.2 约束数据模型：八项，可序列化，可校验

`constraints.Constraints` 是一个只装「这次生成要什么」的 `@dataclass`。它同时承担三件事：构造对话框初值、写进 yaml 供子进程读取、以及运行期只读展示。

字段	默认值	含义	校验 / 提示
scene	scene2	①激光+照片 / ②纯照片	纯照片场景关稠密 → **报错**（没有几何来源）
dense	True	稠密重建开关	纯照片场景必须为真
poisson_depth	8	网格精度（泊松深度 7/8/9）	9 提示面数可能到百万级
voxel	0.02	降采样体素（基准器物米制值）	档位：粗 5 cm / 中 2 cm / 细 1 cm
texture_size	2048	纹理图集边长 1024/2048/4096	4096 提示显存与 4 倍烘焙时间
export_formats	ply,obj,glb	交付格式	空 → **报错**；仅 ply 提示贴图需外挂
quality	True	几何质检（M5）开关	—
force_rerun	False	忽略断点续跑强制重算	勾选后**二次确认**（纯照片一次重算 = 30 分钟起）

这里有一个刻意的分工：`validate()` 只拦**真正的矛盾**（格式为空、档位非法、纯照片关稠密），而「只交 .ply 看不到贴图」「4096 很慢」属于**用户知情的取舍**，走 `warns()` 只提示不拦。理由是：把取舍做成硬错误会逼用户为了跑通而改需求，这不是软件该替用户做的决定。

4.3 闸门：不确认就不启动

`_confirm_constraints()` 是 `_start_run_inner()` 的**第 0 步**，早于任何配置回写与目录创建。两道判断：配置里已有 `constraints_confirmed`（上次确认过）→ 直接用快照，但界面控件要按快照同步（否则控件显示 2048、实际跑 4096）；没有 → 弹 `ConstraintsDialog`。

按钮	语义	后果
开始生成	按当前选择跑	约束写入 cfg 并置 confirmed=True
全部用默认值	不纠结时的快路径	重置为「稠密开 / 精细(8) / 2048² / ply+obj+glb / 质检开」再跑
取消	不确认就不生成	**直接 return，不建目录、不写 yaml、不启动子进程**

> **取消分支为什么要写成用例**：这类闸门最容易被后人改成「点了取消也继续跑」，而它坏了没有任何报错——用户只是偶尔发现「我明明取消了怎么还在跑」。所以用例断言的是 called == []（自检函数未被调用）且 w.runner is None，而不是「没报错」。

4.4 运行期只读：只读卡里不许有输入控件

确认之后，运行期的界面必须表达「这些已经定了」。实现是两步：inputs_host.setVisible(False) 把整个输入区藏起来，再用 ConstraintSummary 显示八行「项目 / 取值」只读摘要。同时「修改约束」按钮禁用、只留「暂停 / 取消」。

只读卡本身也不是「看起来只读」——用例直接在卡里搜四类输入控件：

```
w.constraint_box.set_constraints(Constraints())
for cls in (QComboBox, QCheckBox, QLineEdit, QAbstractSpinBox):
    assert w.constraint_box.findChildren(cls) == []   # 一个都不许有
```

这条断言的意义在于：只要卡里存在任何一个下拉框，用户就会去点它；而运行期改约束既不会生效、也不会报错——那正是上一版「假控件」缺陷的重演路径。

4.5 落盘：子进程只认 yaml

界面里选了不等于生效。确认结果写进 config.yaml 的两处：runner.constraints（八项快照，供溯源与只读展示）与 runner.constraints_confirmed（布尔闸门）。管线本身**不读**这两项——真正的执行参数仍走 sfm_solve / mesh / texture 等原有字段；快照存在的目的是让「这次到底按什么跑的」在事后仍然可查。

从历史项目打开时，to_yaml 用**读改写**而非模板重建：原文件作底稿，只覆盖控件字段，没有对应控件的专业参数用 setdefault 保留。否则打开一个 4096 + 关稠密的历史项目，会被界面默认值悄悄改回 2048 + 开稠密。

五、缺陷③：阶段耗时只活在报表里

5.1 两条数据源，互补使用

「每个阶段各花了多久」这件事，界面原本完全没有位置：30 分钟的运行里只有一根总进度条，结束之后想复盘「是 S1 稠密贵还是 M4 贴图贵」只能翻报表。本版让阶段耗时**常驻**在总时长旁边，数据来自两处：

来源	时机	优点	缺点
GUI 侧墙钟累计	运行中，看到「▲ S 开始 / √ S 完成」就累计	即时可见，用户当下就知道哪一步慢	只统计看得见的那一段，与子进程内计时有小误差；跳过/未启用的阶段无事件
run_manifest 的 elapsed_s	运行结束时覆盖	**权威值**，由批处理进程自己计时，与报表「耗时」列同源	只有跑完才有；读到才算数（文件缺失或损坏时保持实时值并静默降级）

覆盖顺序是有讲究的：**运行后期望看到的数字必须与报表一致**，所以结束时一律用 manifest 覆盖实时值，并在耗时条下方写一行脚注「各阶段合计 01:04 · 1 件 · 总时长 01:09（耗时取自 run_manifest，与报表同源）」。若两者不一致，用户会立刻怀疑软件算错了——而实际上只是口径不同。

5.2 显示：十枚 chip，5 列 × 2 行

`StageTimingStrip` 为每个阶段准备一枚「代号 + 耗时」的小标签（`S1 00:51`），未运行显示 `S1 —`，按状态着色（等待 / 运行中 / 完成 / 失败 / 跳过）。三个设计取舍：

① **不做进度条**：阶段耗时不是百分比，条形图会让人误读成「还差多少」；② **运行中与结束后都保留**：结束后清零就失去了「刚才哪一步慢」的信息，而用户看报表之前多半先看界面；③ **每枚标签宽度由文字自然决定**（最长 `X2 1:02:03`），**不随运行变化**——这是缺陷①的教训：会变的文字不许参与宽度计算。

5.3 口径：同一阶段多件求和，与墙钟分开标注

多件批处理时，同一阶段会在每件文物上各跑一次。用户问的是「S1 这个阶段一共花了多久」，所以**求和**；`total` 同样求和，因此会略大于墙钟总时长（串行时两者相等，只有 `workers>1` 并发时 total 才是累计 CPU 时间）。界面上把「各阶段合计」与「总时长」分开标注，避免用户拿两个数硬对。

另外修掉了一个只在多件批处理下才出现的缺陷：`runner.py` 原本对每个阶段**只发一次**事件（去重），于是第 2 件起没有 `running` 事件，实时耗时只记到第一件。去重去掉之后，累计才对得上。

演示项目（1 件文物、纯照片场景）的实测值，与报表完全一致：

阶段	耗时	状态	说明
S1	**51.78 s**	done	COLMAP 稀疏 + 稠密两遍，本项目的耗时主体
M1	0.11 s	done	去噪 / 法线定向
M2	0.00 s	done	本件无激光锚点，配准为空操作
M3	1.06 s	done	泊松重建（n_threads=1，保可复现）
M4	**11.06 s**	done	UV 展开 + 逐纹素光栅化 + 多视图采样 + 图集编码
M5	0.22 s	done	几何与纹理质检

阶段	耗时	状态	说明
M6	0.07 s	done	导出交付物 + 报表 + 元数据
P1/X1/X2	—	disabled	配置关闭，进「未启用」而不是「未执行」
合计	**64.31 s**	—	墙钟 69 s（差值 = 进程启动与件间开销）

六、缺陷④：目录太散

6.1 旧布局的问题不是丑，是不知道该给谁

旧布局把一切都平铺在项目根：00_raw、10_dense、30_dense、50_tex、90_meta、99_runs、output、80_export、report……十几个同级目录，用户要交付时得自己在里面挑。问题不在于难看，而在于**两个不同用途的东西混在一层：可重算的中间态与要交出去的成果。**

新布局按「能不能直接拿走」切，而不是按阶段号：

```
综合项目/                    <- 用户选定的项目根（一个文件夹搞定）
|-- config.yaml               <- 本次运行的完整配置（含约束快照）
|-- 输出说明.txt              <- 这个目录里什么是什么（给人看的）
|-- 阶段成果/                 <- ① 可重算的中间态，全部在这里
|   |-- 00_raw/  00_photo/  00_ctrl/
|   |-- 10_clean/  20_registered/  30_dense/
|   |-- 50_tex/  60_qc/  70_report/  90_meta/
|   |-- 99_runs/    （SfM 临时目录、日志）
|   `-- output/     （run_manifest.json — 阶段耗时与状态的真源）
`-- 交付成果/                 <- ② 直接交付：双击就能打开
    |-- report/             <- 单一 report 目录（html/csv/png/md）
    `-- <文物编号>/          <- <编号>.ply / .obj / .mtl / _diffuse.png / .glb
```

> **一句话判据**：交付成果 里每个文件都能双击打开（模型、贴图、报表）；阶段成果 里都是删掉可以重算的。用户要交付，只需要进交付成果那一棵；要排查问题，才进阶段成果。

6.2 一行代码的代价：让 task.root 下沉一级

阶段代码里有 200 多处 task.root/阶段目录 的路径拼接。如果逐个改成 task.root/阶段成果/...，会散进几十个文件且以后改不动。实际做法是让 **task.root 的含义从「项目根」下沉为「阶段成果」**——数据根的父目录由 00_raw 的位置决定（discover_items），于是所有阶段代码的路径拼接**自动**跟着走，改动集中在一处。

代价是「项目根」这个概念不再由 task.root 代表。为此新增 src/h3d/core/paths.py 作为布局的**唯一出口**：GUI、CLI、runner、report 一律不许自己拼路径，只能调它。

函数	作用
stage_root / delivery_root	两棵子树的根
project_root_of / is_stage_root	在「项目根」与「阶段成果」之间换算
delivery_item_dir	交付目录：新布局走 交付成果/<编号>，旧布局回退 80_export/<编号>
report_dir / report_html / find_report	报表位置（新老布局都试）
model_candidates	可预览模型回退链：交付成果 → 30_dense → 10_dense → 99_runs/dense
looks_like_project / migrate_legacy	旧布局识别 + 就地迁移（带闸门，见第七章）
ensure_layout / write_layout_readme	建骨架 + 写中文说明（BOM + CRLF，记事本可读）

6.3 兼容：判据不能是「00_raw 在不在」

旧项目必须还能跑。`delivery_item_dir` 的判断是「这一层是不是叫 `阶段成果`，或项目根下是否已有 `阶段成果` / `交付成果`」，是就按新布局，否则按旧布局回退。

这里踩过一个坑：最初用「`00_raw` 是否存在」来判断新老布局，结果**首次运行时 `00_raw` 可能还不存在**，于是一个新项目被误判成旧布局、产物又写回旧路径。判据必须用「只在新布局下才会出现的东西」，而不是「新旧都可能没有的东西」。

迁移只在 **GUI 打开历史项目**时做，CLI 不强制：命令行用户可能用脚本按旧路径（`80_export/...`）取产物，强制改名会直接打断他们的流程。迁移本身是同卷 `shutil.move`（即 rename，465 MB 的项目也不会有明显耗时），同名冲突**跳过并记录**，绝不覆盖用户数据。

6.4 实测目录树与体积

演示项目「综合项目」跑完整管线的实际产物（1 件文物，纯照片场景）：

路径	体积	内容
阶段成果/	26 MB	12 个阶段子目录，含 SfM 临时目录与全部中间态
交付成果/demo_relic/	5.7 MB	ply 0.9 MB · obj 2.1 MB · mtl · glb 1.6 MB · 图集 png 1.2 MB · 2 个 csv
交付成果/report/	84 KB	report.html · metrics_summary.csv · stage_timings.png
输出说明.txt	—	中文说明这个目录里什么是什么

报表首屏的终态行是「完成 7 失败 0 跳过 0（另有 1 个阶段被配置关闭，与本次结论无关）」，分母按已启用阶段算——这一口径是上一版（r13）刚统一的，本版没有回退。

七、两起事故复盘（本版顺手修掉）

7.1 事故 A：迁移逻辑吃掉了仓库的 output 目录

现象：跑完一轮测试后 git status 出现 **167 个 D**（output/ 下全部文档产物），看起来像测试把 output 删了。真因不是删，是**搬**：某个环节把存放项目的目录当成了项目根，migrate_legacy 把 output/ 整棵 rename 进了阶段成果/output。

根因在判据：legacy_dirs_present() 把 output 与 report 也算作旧布局痕迹，而这两个名字太通用——**任何仓库、任何用户目录都可能有一个叫 output 的文件夹**。把它们当作「这是 H3D 项目」的证据，等于给误伤开了门。

修法：新增 looks_like_project()，判据收紧到**项目专属痕迹**——config.yaml 或任一阶段专属目录（00_raw / 00_photo / 30_dense / 80_export 等）。output 与 report **单独出现不算数**。闸门设在两处：legacy_dirs_present 直接返回空，migrate_legacy 自己也拒绝（返回 refused 原因，GUI 记 WARNING）。另加回归用例：只含 src / output / report / docs 的目录**一个文件都不许挪走**。

7.2 事故 B：PySide6 在无 windowHandle 时询问 screen 会堆损坏

现象：全量测试跑至第 188 个用例**无声硬崩**，退出码 127，无 Python traceback；单独跑该文件却全绿。用 faulthandler 抓到：

```
Windows fatal exception: code 0xc0000374      <- STATUS_HEAP_CORRUPTION
Current thread 0x000026e0 (most recent call first):
  File "src/h3d/gui/app.py", line 121 in _fit_to_screen
  File "src/h3d/gui/app.py", line 78 in __init__
  File "tests/test_gui_constraints.py", line 190 in test_confirmed_skips_dialog
```

定位过程值得记一下：崩溃点与哪个用例无关，只与前面跑过多少东西有关——单文件绿、两两组合绿、三个文件一起必崩。这是**堆损坏的典型特征**：越界写发生在别处，检测发生在下一次堆操作。加了 RSS 探针记录每个用例的内存与句柄数，确认**不是内存耗尽**（崩溃时 329 MB），才转向追谁损坏了堆。

真凶是 _fit_to_screen() 里的 self.screen()：窗口在构造期还没有 windowHandle，PySide6 6.11 下这个调用会在堆上留下破坏。修法是不再在构造期问 widget 自己，改成「有 windowHandle 才问它，否则直接问 QGuiApplication.primaryScreen()」。改后同一组合连跑 2 次全绿，全量 **843 项**通过。

> **可复用的一条经验**：**单跑绿、合跑崩、无 traceback = 先查内存，不要查逻辑。**把时间花在「哪个用例污染了全局变量」上是白费——因为污染根本不在 Python 层。正确的第一步是 faulthandler 拿到原生栈、第二步是给每个用例打 RSS / 句柄探针排除 OOM，第三步才是按组合二分。这三步走完，两个事故都在一小时内定位。

7.3 事故 C：全跳过的件卡在 RUNNING，界面误报「运行未完成」

这是打包版端到端复跑时抓到的，属于**真机复核的收益**——单测全绿，但把打包版对着已经跑完的项目再跑一次（全部阶段断点续跑跳过），汇总行却是 total=1 done=0 failed=0 skipped=0。

三个计数全是 0，意味着界面判据 done+skipped>0 为假 → 弹「运行未完成」。而这正是 1.1.7（r11）修过的同一类误报，只是换了一条到达路径。用状态迁移探针打出全过程：

```
restore_stage  S1  PENDING  -> SKIPPED
restore_stage  M1  SKIPPED  -> SKIPPED
...
restore_stage  M4  SKIPPED  -> SKIPPED
```

```
begin_stage   P1  SKIPPED  -> RUNNING    <-- 先打「P1 开始」
mark_disabled  P1  RUNNING  -> RUNNING    <-- 再判定未启用，状态没收回
restore_stage  M5  RUNNING  -> SKIPPED
restore_stage  M6  SKIPPED  -> SKIPPED
begin_stage   X1  SKIPPED  -> RUNNING    <-- 同样的问题，最后一个阶段是 X2
mark_disabled  X2  RUNNING  -> RUNNING
FINAL TaskStatus.RUNNING                <-- 任务终态停在 RUNNING
```

根因：可选阶段（P1 / X1 / X2）的调度是**先 `begin_stage` 打开始日志、后**才判定「配置里没启用」。而 `mark_disabled` 的设计意图是「不动任务级状态」（否则跑完 M1~M6 的件会因为 P1 关着而变成 SKIPPED），于是那次 RUNNING 就再没人收回。最后处理的恰好是未启用的 X2，任务就停在那里。

修法两层，缺一不可：① `begin_stage` 记下进入前的状态，`mark_disabled` 在任务仍是 RUNNING 时还原它——修掉这条具体路径；② 件级终态**收敛函数** `_finalize_task_status()`，按阶段结果逆推（有 DONE → DONE；有 FAIL → FAIL；有 RUNNING 残留 → FAIL；其余 → SKIPPED），主流程与并行 worker 返回前都调一次——堵死所有「循环跑完但状态没落定」的将来路径。

> **为什么第②层不能省**：第①层修的是「我已知的那条路径」。而这类缺陷的本质是**状态机有入口没出口**——只要还有人能在不收敛的情况下返回，就会再长出一个新入口。终态收敛函数的价值不在于它今天修了什么，而在于它保证**「RUNNING」这个中间态永远不会出现在结果里。**同理，宁可把残留的 RUNNING 收敛成 FAIL 也不收敛成 DONE：误报失败只会让人多点一次，误报成功会让人以为交付物已经好了。

验证：新增 5 项用例（`test_task.py`）——未启用阶段回退状态、跑完后关闭 P1 仍是 DONE、全跳过收敛为 SKIPPED、DONE 优先于 SKIPPED、RUNNING 残留不泄漏。改后同一项目复跑：`done=0 skipped=1`，界面判「无需重跑（全部已是最新）」。

八、验证

8.1 单元测试：843 项全绿

全量 pytest 结果 **843 passed**（102.5 s），其中本版新增 65 项：布局 24 项、约束 20 项、窗口 15 项、件级终态收敛 5 项、结果条旧布局兼容 1 项。另外改动 4 个既有测试文件（CLI 初始化断言改新布局、结果条两例、conftest 增加「模态框自动确认」会话级 fixture）。

conftest 的这条 fixture 本身也是一个坑的产物：ConstraintsDialog.exec() 是真模态，在 offscreen 测试里没人点按钮，它会**永久阻塞**整个 pytest 进程（实测把全量测试挂死在 20 分钟超时）。这属于测试环境约定，不改被测行为——需要验证取消分支的用例自己 monkeypatch 成返回 0。

8.2 端到端：CLI 跑通 + 目录树核对 + 报表口径

在 D:\h3d_e2e_layout\综合项目 上跑 h3d.cli batch，69 s 完成。核对项：① 日志里所有路径均为新布局；② 三处出口（M6 导出、报表、S1 临时目录）都在正确子树；③ 报表首屏终态为「完成 7 / 失败 0 / 跳过 0 / 未启用 1 / 未执行 0」；④ 交付目录里 ply / obj / mtl / glb / 图集 / csv 齐全（r13 修的图集无条件落盘没有回退）。

8.3 发行包

版本号唯一来源（h3d.__version__ = 1.1.10）驱动全链路：版本资源 → exe 的 PE FileVersion → 安装包属性页 → VERSION 清单 → 发行包文件名，任一环节不一致都会在打包前被校验拦住。发行包内容与上一版结构一致：H3D.exe + Setup + README + VERSION（SHA-1）+ configs/ + docs/ + source/ + tools/。

九、已知限制与下一步

1) **本版未在真机上验证窗口行为。**尺寸稳定性有成套断言（文字换 240 倍长串、进度条格式串守卫），但「用户拉着窗口缩放后再跑」这条路径没有用例覆盖——下一步应加「resize 之后再喂长文本」的用例，并在 1366×768 / 150% 缩放的真实屏幕上目视一次。

2) **约束与执行参数仍是两套字段。**runner.constraints 是快照，真正执行走 sfm_solve / mesh / texture。本版由 apply_to_project_config() 保证两者同步，但若有人手改 yaml 只改了快照，界面会显示一个不存在的约束。下一步考虑给快照加来源标记（界面确认 / 手改 / 迁移），并在加载时比对不一致。

3) **阶段耗时的实时值在跳过阶段上是空的。**跳过（断点续跑命中）没有「▲ 开始 / √ 完成」事件，只能等结束时由 manifest 的 elapsed_s 补上。运行中看到 M3 — 并不等于没跑，这一点目前只写在脚注里，下一步在 chip 上加「跳」角标。

4) **迁移是单向的**：阶段成果 / 交付成果 → 旧布局 没有反向工具。若用户误迁（例如把文档目录当成项目根），只能手工搬回。本版加的 looks_like_project 闸门已能拦住绝大多数误伤，但「确实像项目、其实不是」的情况仍需人工判断——迁移日志里记了每一条 moved / skipped，可以照着搬回。

5) **多件批处理的耗时口径待实测。**求和逻辑有单测，但 workers>1 并发下「各阶段合计」会大于墙钟总时长，界面上已分开标注，尚未在真实多件项目上核对数值。

> **本版可复用的一条工程经验**：**能跑完和用户看得懂是两件事。**这一版没有改一行算法，四条抱怨全部来自算法之外——窗口、参数、时间、目录。而这四项里，**三项的根因都是界面在展示变量、却没有约束变量的尺寸 / 来源 / 时机**：文字长度撑宽窗口、参数随改随生效、耗时只活在报表里。凡是界面上会变的东西，都必须同时回答三个问题——**它变的时候会不会挤动别人？它变了算不算数？它不变的时候还能不能看到？**